

The background is a teal color with a white circuit board pattern. A diagonal white line runs from the top-left corner to the bottom-right corner, dividing the image into two triangular sections.

PATRONES DE DISEÑO

OBJETIVOS

- ① Conocer cuáles son los patrones de diseño de comportamiento.
- ① Identificar cuándo aplicar cada uno de los patrones de diseño de comportamiento.

Dennis S. Cohn Murov, Mag. Ing.
Pontificia Universidad Católica del Perú



1

PATRONES DE DISEÑO DE COMPORTAMIENTO

Detallemos algunos de los patrones ...

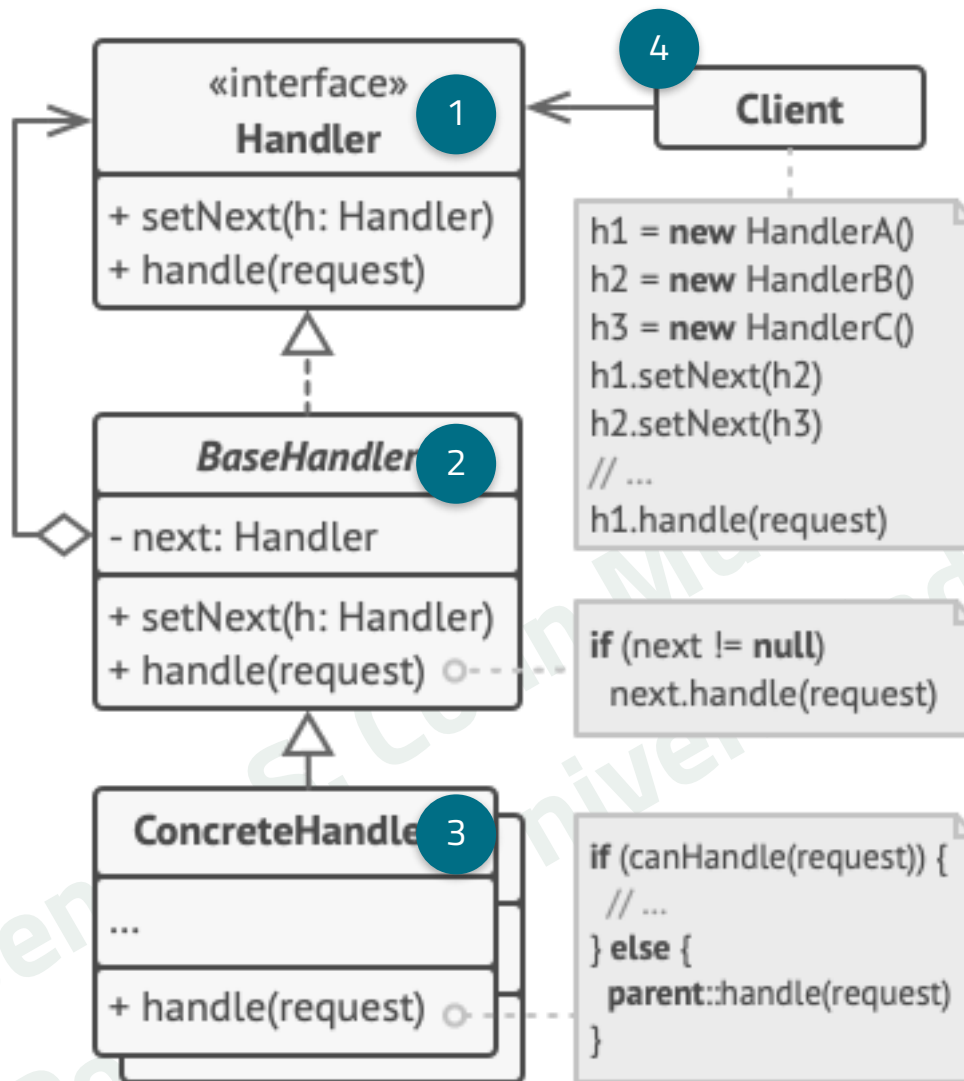
PATRÓN: CHAIN OF RESPONSIBILITY

PROBLEMA QUE CUBRE

- ① Más de un objeto (no conocido a priori) puede gestionar una solicitud.
 - ② No se desea especificar al manejador de forma explícita.
 - ③ El listado de manejadores debe ser dinámico.

¿CÓMO LO HACE?

Permite pasar solicitudes a lo largo de una cadena de manejadores. Cada manejador decide si procesa la solicitud o si la pasa al siguiente manejador de la cadena.



- 1** Interfaz para los Manejadores. Tiene un método para atender la solicitud y otro para remitirla a otro manejador.
- 2** Manejador Base Abstracto. Incluye la referencia al siguiente manejador.
- 3** Manejador Concreto contiene la lógica para atender la solicitud (decide si la atiende y/o la pasa al siguiente manejador).
- 4** Cliente arma la cadena de manejadores (estática o dinámicamente) y le envía la solicitud.

CONSECUENCIAS: CHAIN OF RESPONSIBILITY

- ⦿ (+) Permite controlar el orden de control de solicitudes.
- ⦿ (+) Permite introducir nuevos manejadores en la aplicación sin descomponer el código cliente existente.
- ⦿ (-) Impacto en el tiempo de ejecución y recursos de procesamiento; al tenerse objetos instanciados que solo reenviarán la solicitud.

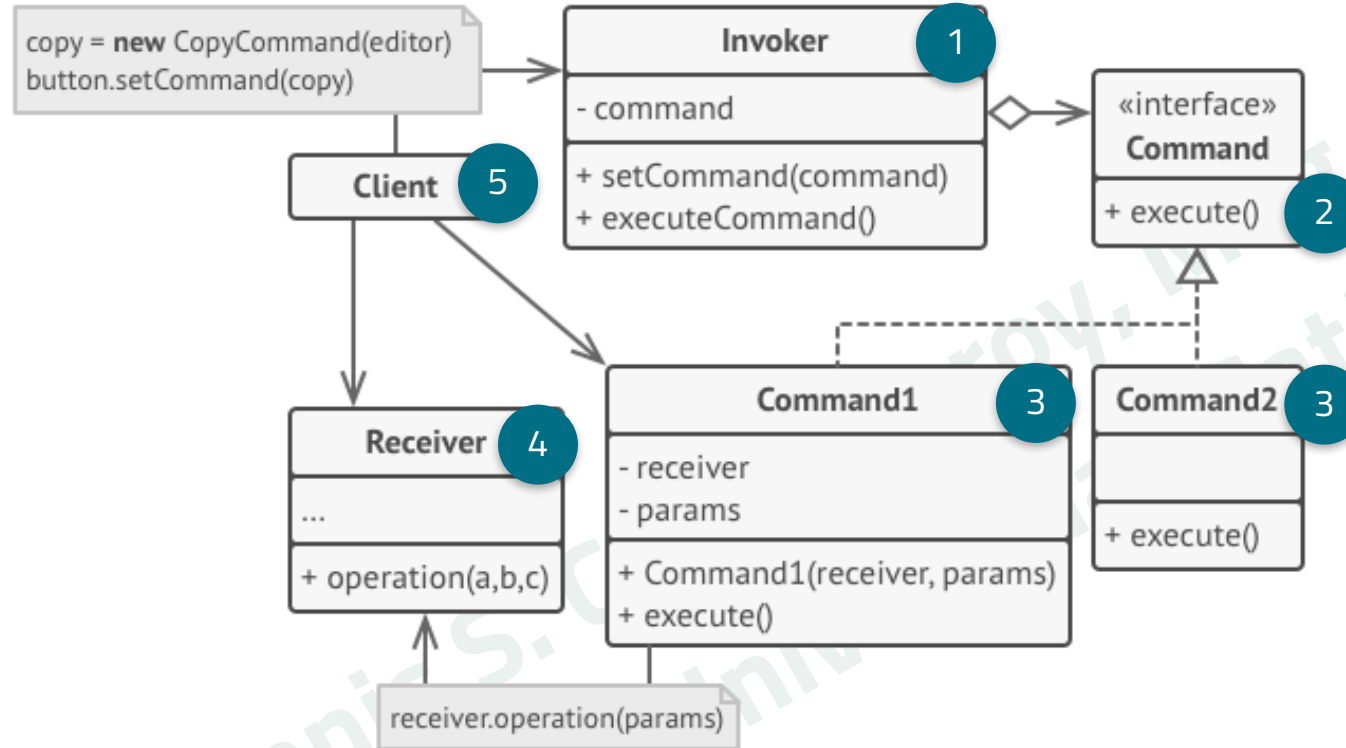
PATRÓN: COMMAND

PROBLEMA QUE CUBRE

- Encolar operaciones o ejecutarlas de forma remota.
- Parametrizar objetos con operaciones.
- Soportar deshacer.
- Contar con datos de trazabilidad de una operación.

¿CÓMO LO HACE?

Transforma solicitudes (requests) en objetos que contienen toda la información de la solicitud.



- 1 Invocador inicializa las solicitudes. Almacena una referencia a un objeto comanda. Llama a este objeto en lugar de enviar la solicitud al receptor.
- 2 Normalmente solo cuenta con un método `execute()`.
- 3 Los parámetros son atributos del objeto. Arma la llamada para la lógica de negocio.
- 4 Objeto que recibe el comando. Ejecuta el trabajo.
- 5 Crea y configura el objeto Comanda (parámetros y Receiver). Asocia la Comanda con el Invocador.

CONSECUENCIAS: COMMAND

- ⦿ (+) Desacopla las clases que invocan operaciones de las que realizan esas operaciones.
- ⦿ (+) Permite introducir nuevos comandos en la aplicación sin descomponer el código cliente existente.
- ⦿ (+) Permite implementar deshacer/rehacer
- ⦿ (-) Mayor complejidad en el código; se tiene una nueva capa (el invocador) entre el emisor y el receptor.

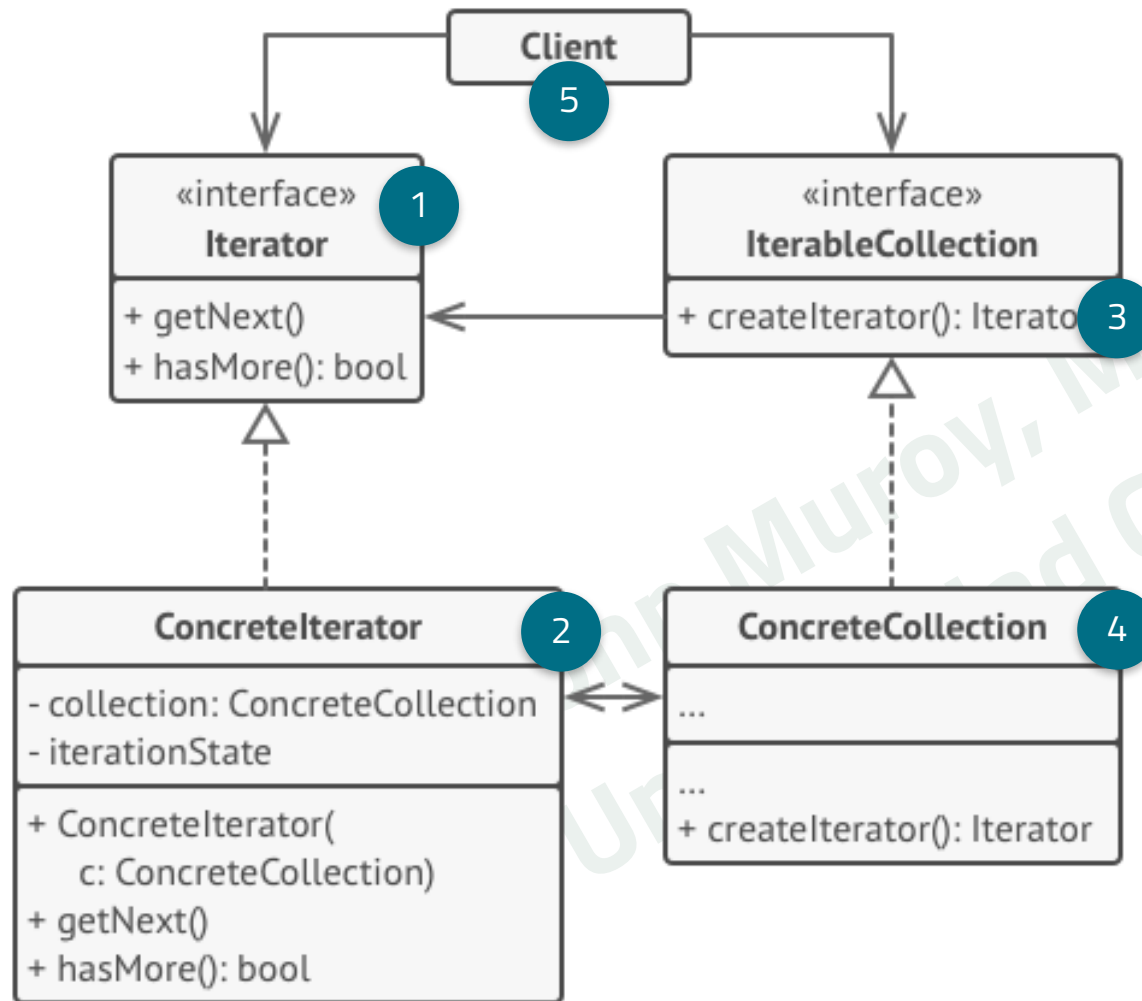
PATRÓN: ITERATOR

PROBLEMA QUE CUBRE

- ⦿ Mantener una estructura compleja (y privada por conveniencia o seguridad).
- ⦿ Reduce duplicación al implementar iteradores.

¿CÓMO LO HACE?

Permite recorrer los elementos de una colección sin exponer su estructura (lista, árbol, pila, etc.).



- 1 Declara los métodos para recorrer una la colección.
- 2 Implementa los métodos para recorrer la colección.
- 3 Declara el método para obtener un iterador compatible con la colección.
- 4 Implementa la colección y el método para obtener un iterador compatible.
- 5 Trabaja con el iterador y la colección a través de sus interfaces a fin de no generar acoplamiento.

CONSECUENCIAS: ITERATOR

- ⦿ (+) Las estructuras y sus iteradores tienen clases dedicadas.
- ⦿ (+) Permite reemplazar las colecciones e iteradores.
- ⦿ (-) No se recomienda su uso en caso se trabaje con colecciones sencillas.

Dennis S. Cohn Murray, Mag. Ing.
Pontificia Universidad Católica del Perú

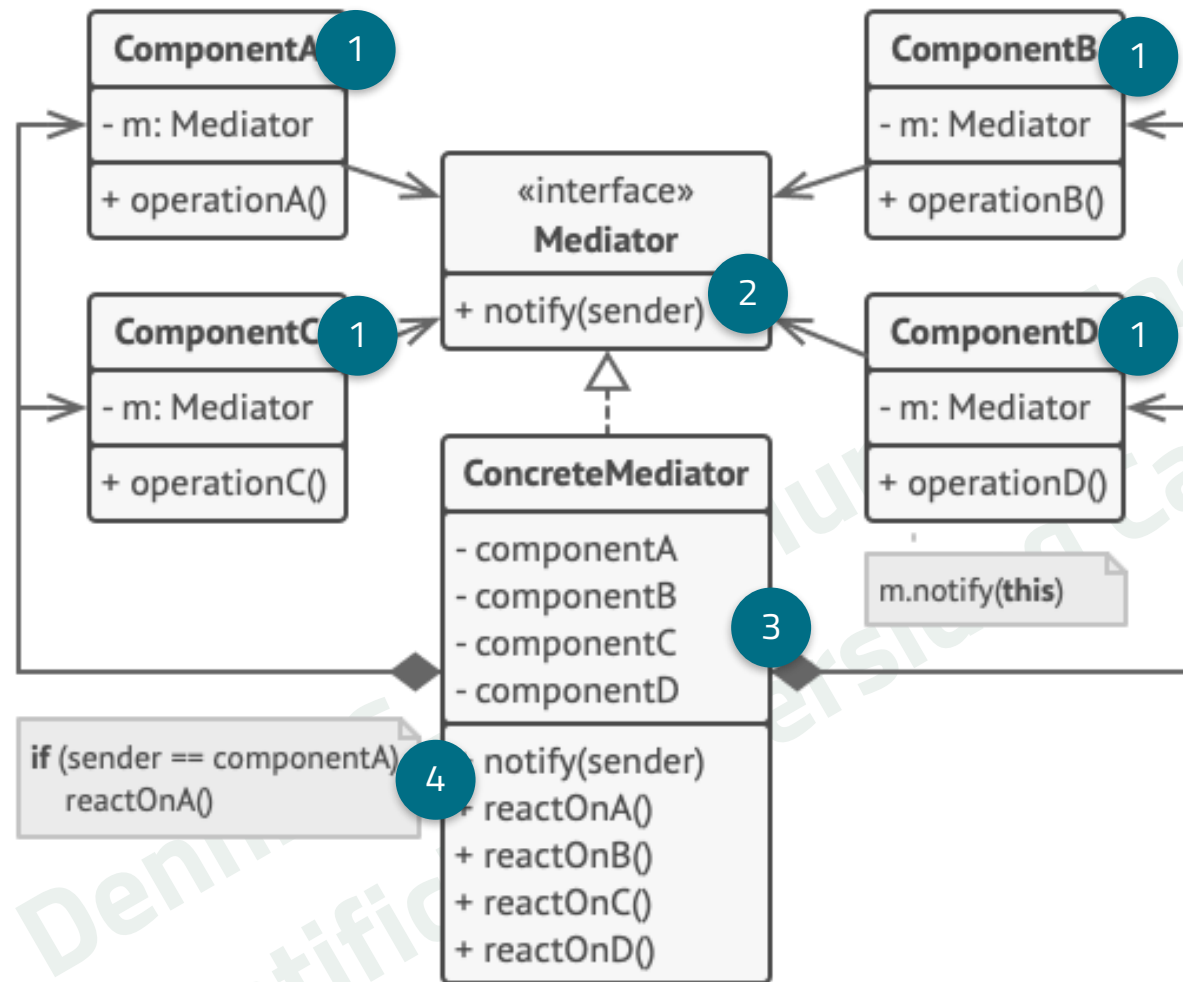
PATRÓN: MEDIATOR

PROBLEMA QUE CUBRE

- ① Un conjunto de objetos se comunica de forma compleja; ello impacta en la legibilidad de la solución.
- ① La reutilización de objetos es compleja dado su fuerte acoplamiento.

¿CÓMO LO HACE?

Reduce las dependencias caóticas entre objetos. Restringe las comunicaciones directas entre los objetos al utilizar un objeto mediador.



1 Componentes que contienen lógica de negocio. Referencia a la interfaz mediadora.

2 Interfaz que declara métodos de comunicación con componentes. Normalmente incluyen un único método de notificación.

3 Mediadores Concretos encapsulan las relaciones entre varios componentes.

4 Los componentes no deben conocer otros componentes. Cada componente sólo debe notificar a la interfaz mediadora. Cuando la mediadora recibe la notificación, debe poder identificar fácilmente al emisor.

CONSECUENCIAS: MEDIATOR

- ⦿ (+) Permite extraer las comunicaciones entre varios componentes dentro de un único sitio; facilitando el mantenimiento.
- ⦿ (+) Permite introducir nuevos mediadores sin tener que cambiar los propios componentes.
- ⦿ (-) Un mediador podría evolucionar en un objeto complejo y crítico.

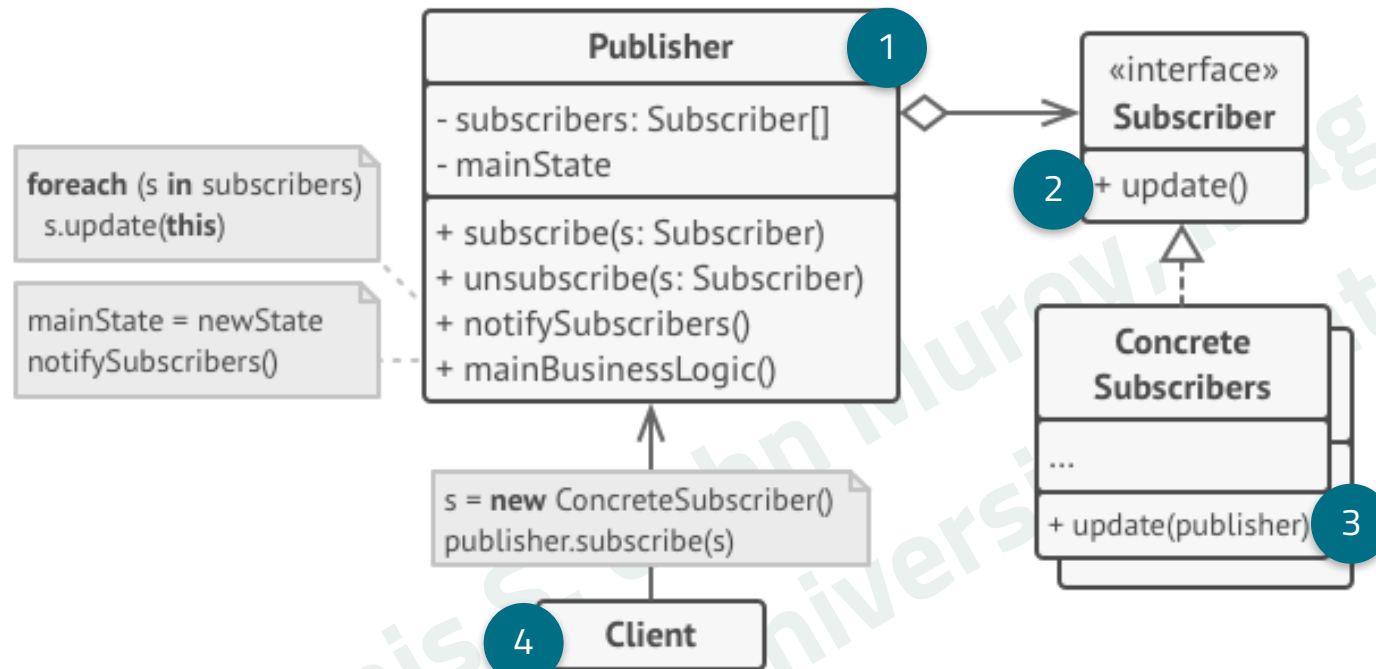
PATRÓN: OBSERVER

PROBLEMA QUE CUBRE

- Cuando un cambio de estado en un objeto debe de ser notificado a otros objetos (desconocidos o dinámicos).
- Cuando un objeto requiere observar a otro bajo condiciones determinadas.

¿CÓMO LO HACE?

Define un mecanismo de subscripción a un objeto. Cualquier cambio de estado sobre el objeto es remitido a los objetos suscritos.



- 1 Envía eventos de interés a otros objetos. Permite a los objetos suscribirse.
- 2 El publicador invoca el método de notificación de cada suscriptor.
- 3 Los subscriptores efectúan alguna acción en base a la información recibida desde el publicador.
- 4 Durante la inicialización, crea los objetos Suscriptores y el objeto Publicador; y los asocia.

CONSECUENCIAS: OBSERVER

- ⦿ (+) Permite introducir nuevas clases subscriptoras sin tener que cambiar el código de la publicadora.
- ⦿ (-) La recepción de las notificaciones por parte de los suscriptores es asíncrono. No hay un control en el orden en que estas notificaciones son recibidas por cada objeto subscriptor.

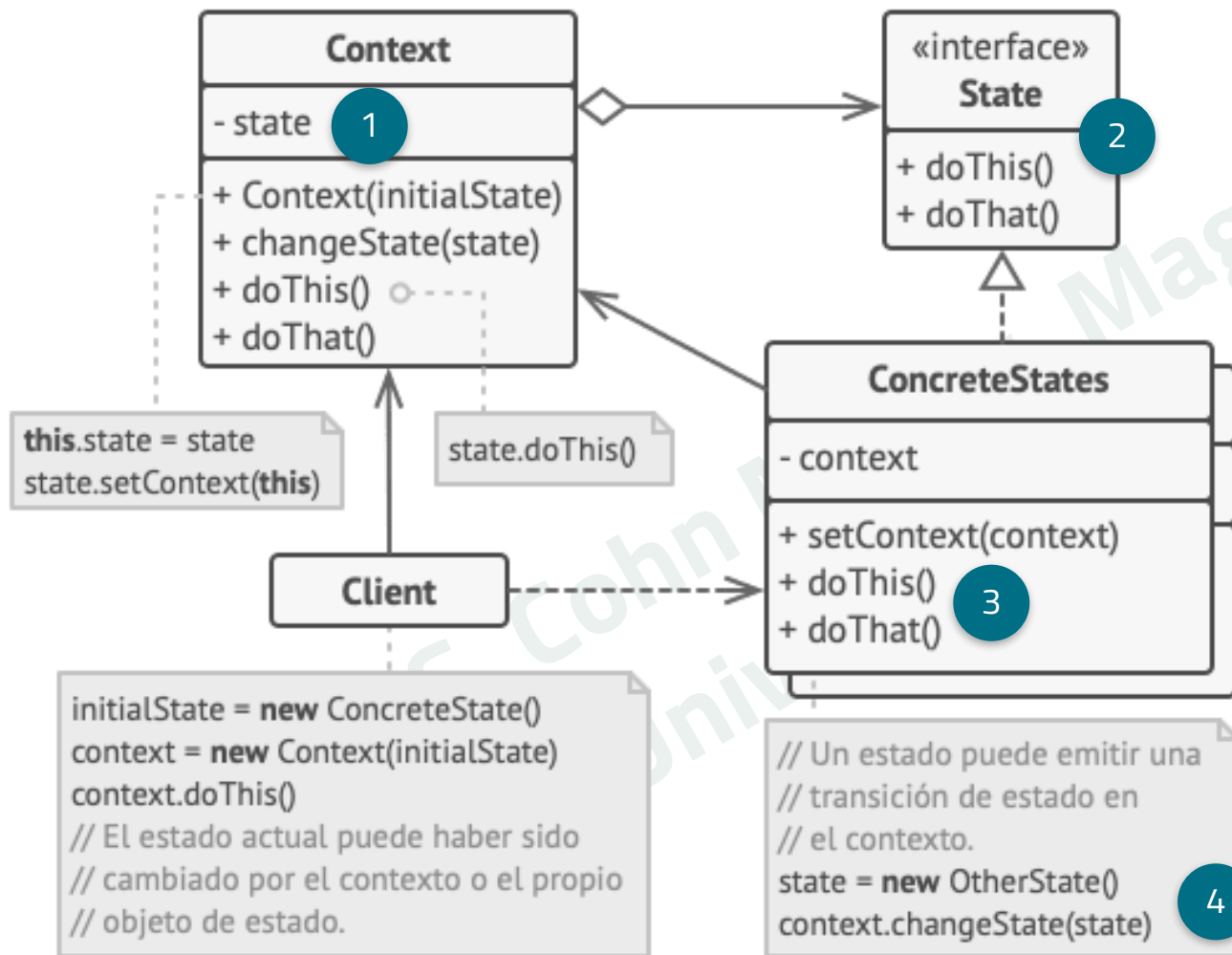
PATRÓN: STATE

PROBLEMA QUE CUBRE

- ⦿ El comportamiento de un objeto debe depender de su estado.
- ⦿ Estructuras condicionales complejas relacionadas al estado del objeto.

¿CÓMO LO HACE?

Permite a un objeto alterar su comportamiento cuando su estado interno cambia.



- 1 Clase cuyo comportamiento cambia según su estado. Almacena una referencia a un objeto de estado y le delega el trabajo específico del estado
- 2 La interfaz Estado declara los métodos comunes para todos los estados.
- 3 Implementa los métodos específicos del estado. Pueden almacenar una referencia al Contexto para extraer datos requeridos.
- 4 Pueden efectuar un cambio de estado del Contexto.

CONSECUENCIAS: STATE

- ⦿ (+) Organiza el código relacionado a los estados en clases separadas.
- ⦿ (+) Introduce nuevos estados sin cambiar clases de estado existentes o la clase contexto.
- ⦿ (+) Elimina estructuras condicionales complejas.
- ⦿ (-) No se recomienda su uso si se manejan pocos estados o si estos cambian con poca frecuencia.

2

REFERENCIAS

BIBLIOGRAFÍA

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design patterns: elements of reusable object-oriented software. Pearson Deutschland GmbH.
- Freeman, E., & Robson, E. (2020). Head First Design Patterns. O'Reilly Media.
- Design patterns and refactoring. (n.d.). https://sourcemaking.com/design_patterns

Créditos:

- Plantilla de la presentación por [SlidesCarnival](#)
- Fotografías por [Unsplash](#)
- Diseño del fondo [Hero Patterns](#)